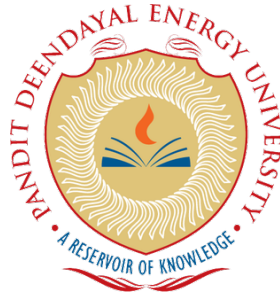

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
SCHOOL OF TECHNOLOGY
PANDIT DEENDAYAL ENERGY UNIVERSITY
SESSION 2024–25



PixelPM

Project Management Application

Technical Architecture & Cloud Services Report

SUBMITTED BY

NAME : Jeet Patel
ROLL NO. : 23BIT156
NAME : Manav Patel
ROLL NO. : 23BIT179
NAME : Bhumik Verma
ROLL NO. : 23BIT199
COURSE : Cloud Architecture and Services
NAME
COURSE CODE : 20IC316T

SUBMITTED TO

Dr. Deepak Sharma

Assistant Professor

Department of Information and Communication Technology
Pandit Deendayal Energy University

Contents

1	Application Overview	3
1.1	What Is PixelPM?	3
1.2	Target Audience	3
2	Architecture Overview	5
2.1	High-Level Architecture	5
2.2	Production Request Flow	5
2.3	Local Development Flow	5
3	System Diagram	7
4	AWS Free Tier Cloud Services	8
5	Service Integration	10
5.1	CloudFront + S3 — Static Frontend Delivery	10
5.2	CloudFront + API Gateway — API Routing	10
5.3	API Gateway + Lambda — Serverless Compute	10
5.4	Lambda + DynamoDB — Data Persistence	10
5.5	Lambda + IAM — Least-Privilege Access	11
5.6	Lambda + CloudWatch — Observability	11
5.7	AI Chat — Ollama Integration (Local Development)	11
6	Application Screenshots	12
6.1	Dashboard — Board View (Kanban)	12
6.2	Dashboard — List View	13
6.3	Dashboard — Overview	13
6.4	Create Project Modal	14
6.5	Task Detail Modal	15
6.6	AI Chat Widget	16
7	Code Overview	17
7.1	Express Server Entry Point (<code>backend/src/app.js</code>)	17
7.2	DynamoDB Client Configuration (<code>backend/src/db/dynamodb.js</code>)	18
7.3	Tasks Route with GSI Query (<code>backend/src/routes/tasks.js</code>)	19
7.4	AI Route with Workspace Context Builder (<code>backend/src/routes/ai.js</code>)	20
7.5	React Application Component (<code>src/App.jsx</code>)	21
7.6	CloudFormation Infrastructure Template (<code>infrastructure/cloudformation.yaml</code>)	22
7.7	Docker Compose Local Stack (<code>docker-compose.yml</code>)	23
8	Conclusion	24

List of Figures

3.1	PixelPM System Architecture Diagram (Production & Local Development)	7
6.1	Board view: Kanban columns with task cards, priority badges, and team avatars	12
6.2	List view: tabular task display with status, priority, and assignee columns	13
6.3	Overview: project-level completion statistics and team workload summary	13
6.4	Create project modal: form for adding a new project to the workspace .	14
6.5	Task detail modal: editing title, status, priority, and assignee	15
6.6	AI chat widget: Ollama-powered assistant with live workspace context injection	16
7.1	backend/src/app.js: Express server, route mounting, and global error handler	17
7.2	backend/src/db/dynamodb.js: dual-mode DynamoDB client (AWS or local emulator)	18
7.3	backend/src/routes/tasks.js: RESTful task endpoints including GSI-backed project query	19
7.4	backend/src/routes/ai.js: live workspace snapshot builder and Ollama chat proxy	20
7.5	src/App.jsx: centralised React state, view modes, and drag-and-drop handler	21
7.6	infrastructure/cloudformation.yaml: complete IaC definition for all AWS resources	22
7.7	docker-compose.yml: six-container local development stack with health checks	23

1. Application Overview

1.1 What Is PixelPM?

PixelPM is a full-stack, serverless **project management web application** that enables software teams and individuals to organise, track, and collaborate on work in real time. The application provides an intuitive Kanban-style interface alongside list and overview perspectives, letting teams visualise progress across multiple projects simultaneously.

At its core PixelPM solves the problem of scattered task tracking by consolidating projects, tasks, team members, and progress metrics into a single lightweight tool that runs entirely within the AWS free tier, meaning it costs nothing to operate at small scale.

Key capabilities include:

- **Multi-project workspace** — create, manage, and switch between independent projects, each with its own task board.
- **Task lifecycle management** — tasks move through three statuses: *To Do*, *In Progress*, and *Done*, via drag-and-drop or a detail modal.
- **Priority tracking** — tasks carry a priority label (*low*, *medium*, *high*) and can be assigned to specific team members.
- **Three view modes** — Board (Kanban columns), List (tabular), and Overview (project-level summary and statistics).
- **AI assistant** — a floating chat widget backed by a locally-running Ollama llama3.2:3b model that is injected with a live workspace snapshot so it can answer questions about project status, team workloads, and task progress.
- **Team management** — colour-coded user avatars and per-user task assignment across all projects.

1.2 Target Audience

PixelPM is designed for the following user groups:

Small software teams (2–15 people)

Teams that find enterprise tools like Jira or Asana overly complex or expensive for their needs. PixelPM provides the essentials—Kanban boards, priorities, assignments—without the configuration overhead.

Indie developers and freelancers

Solo practitioners managing multiple client projects simultaneously. The zero-cost AWS free-tier deployment means there are no ongoing hosting fees.

Bootcamp students and early-career developers

The open architecture and fully self-hostable local stack (Docker Compose) make it an ideal learning platform to study full-stack development, cloud services, and Infrastructure-as-Code in one coherent project.

Hackathon and side-project teams

Groups that need to spin up a task tracker quickly without provisioning complex infrastructure. A single `docker compose up` starts the full stack locally, and a single CloudFormation deployment creates all AWS resources in under five minutes.

2. Architecture Overview

2.1 High-Level Architecture

PixelPM follows a **serverless, three-tier architecture** split into:

1. **Presentation tier** — React 19 single-page application (SPA) built with Vite, served via Amazon CloudFront and Amazon S3.
2. **API tier** — Express.js RESTful backend running inside a containerised AWS Lambda function, exposed through Amazon API Gateway (HTTP API v2).
3. **Data tier** — Amazon DynamoDB with three tables and one Global Secondary Index (GSI) for efficient project-scoped queries.

All tiers are defined in a single `infrastructure/cloudformation.yaml` Infrastructure-as-Code (IaC) template, making the full environment reproducible and version-controlled.

2.2 Production Request Flow

1. A user's browser loads `https://<cf-domain>` (CloudFront).
2. CloudFront serves static React assets (`index.html`, bundled JS/CSS) from the private S3 bucket using Origin Access Control (OAC).
3. API calls made by React (path prefix `/api/*`) are forwarded to the API Gateway origin with caching *disabled*.
4. API Gateway invokes the Lambda function via `AWS_PROXY` integration, passing the full HTTP request.
5. The Lambda (containerised Express.js) processes the request, executing DynamoDB operations using the AWS SDK v3.
6. Lambda returns a response through API Gateway back to CloudFront, which forwards it to the browser. Execution logs are written to Amazon CloudWatch.

2.3 Local Development Flow

For development, Docker Compose orchestrates five containers:

1. **DynamoDB Local** — in-memory DynamoDB emulator on port 8000.
2. **db-init** — one-shot Node.js container that creates tables and seeds demo data (4 users, 3 projects, 8 tasks).
3. **Ollama** — local LLM server on port 11434.
4. **model-init** — one-shot container that pulls `llama3.2:3b` on first run.
5. **Backend** — Express.js API on port 3001, configured with `DYNAMODB_ENDPOINT`

pointing to the local emulator.

6. **Frontend** — Nginx-served React build on port 8080, proxying `/api` requests to the backend container.

3. System Diagram

The Mermaid diagram in `system_diagram.mmd` (reproduced as a rendered image below) illustrates all services, their relationships, and the two execution environments (AWS production and local Docker).

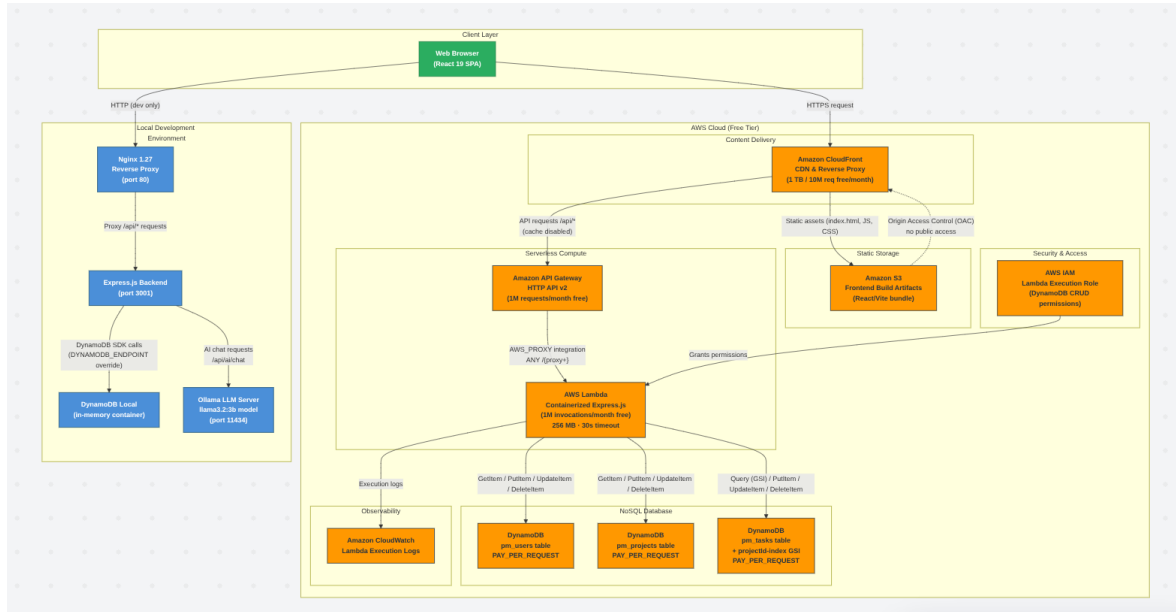


Figure 3.1: PixelPM System Architecture Diagram (Production & Local Development)

The diagram shows three distinct zones:

Client Layer

The user's web browser running the React SPA.

AWS Cloud (Free Tier)

CloudFront CDN, S3 static hosting, API Gateway HTTP API, Lambda compute, three DynamoDB tables, IAM execution role, and CloudWatch logging.

Local Development Environment

Nginx reverse proxy, Express.js backend, DynamoDB Local emulator, and Ollama LLM server—all orchestrated by Docker Compose.

4. AWS Free Tier Cloud Services

PixelPM is engineered to stay within the AWS free tier at small scale. The table below lists every AWS service used, its specific configuration in the project, and the corresponding free-tier allowance.

Service	Usage in PixelPM	Free-Tier Allowance
Amazon DynamoDB	Dy- Three tables: <code>pm_users</code> , <code>pm_projects</code> , <code>pm_tasks</code> . <code>PAY_PER_REQUEST</code> billing mode. Point-in-Time Recovery enabled. GSI <code>projectId-index</code> on <code>tasks</code> table for project-scoped queries.	25 GB storage + 25 read/write capacity units (provisioned) or 200 million requests/month (on-demand) — always free.
AWS Lambda	Single function <code>pixelpm-api</code> : containerised Express.js image from ECR. 256 MB memory, 30-second timeout. Handles all API routes via <code>AWS_PROXY</code> integration.	1,000,000 free invocations/month + 400,000 GB-seconds compute — always free.
Amazon Gateway (HTTP v2)	API HTTP API (<code>pixelpm-api</code>) with a single catch-all route <code>ANY /{proxy+}</code> forwarding to Lambda. CORS configured for all origins, methods, and content-type header. Auto-deploy stage.	1,000,000 HTTP API calls/month free for the first 12 months.
Amazon S3	Private bucket <code>pixelpm-frontend-static</code> stores the Vite build output (HTML, JS, CSS, assets). All public access is blocked; CloudFront accesses the bucket via Origin Access Control (OAC, SigV4 signing).	5 GB storage + 20,000 GET + 2,000 PUT requests/month free for 12 months.

Service	Usage in PixelPM	Free-Tier Allowance
Amazon Cloud-Front	Single distribution with two origins: S3 (default cache behaviour — static assets, CachingOptimised policy) and API Gateway (/api/* behaviour — CachingDisabled policy, all HTTP methods allowed). SPA fallback returns <code>index.html</code> on 403/404. HTTP/2 enabled.	1 TB data transfer out + 10,000,000 HTTP/HTTPS requests/month — always free.
AWS IAM	Role <code>pixelpm-lambda-role</code> with managed policy <code>AWSLambdaBasicExecutionRole</code> (CloudWatch logs) plus an inline policy granting DynamoDB CRUD operations on all three tables and the tasks GSI.	IAM has no cost — always free.
Amazon Cloud-Watch	Lambda execution logs streamed automatically via the managed policy. Useful for debugging cold-start times, DynamoDB errors, and Ollama timeouts.	5 GB log data ingestion + 5 GB log storage/month free for 12 months.

Table 4.1: AWS Free-Tier Services Used in PixelPM

5. Service Integration

5.1 CloudFront + S3 — Static Frontend Delivery

The React application is compiled by Vite into a static bundle and uploaded to an S3 bucket with all public access blocked. CloudFront is the only entity that can read from S3, enforced by an **Origin Access Control** resource that signs every S3 request with SigV4. A bucket policy explicitly allows `s3:GetObject` only when the requesting `AWS:SourceArn` matches the CloudFront distribution ARN.

The distribution's default cache behaviour targets the S3 origin and applies the AWS managed `CachingOptimised` policy, providing long-lived browser caching for hashed asset filenames. A **SPA fallback** is configured by mapping both HTTP 403 and 404 error codes to a 200 response returning `/index.html`, which is required for React Router client-side navigation to work on direct URL access or page refresh.

5.2 CloudFront + API Gateway — API Routing

A second CloudFront cache behaviour matches the `/api/*` path pattern and routes those requests to the API Gateway origin using the `CachingDisabled` managed policy (no caching, all HTTP methods forwarded). This means the React frontend makes all API calls to its own CloudFront domain with no CORS issues; CloudFront transparently proxies them to API Gateway over HTTPS.

5.3 API Gateway + Lambda — Serverless Compute

The HTTP API uses a single `AWS_PROXY` integration pointing to the Lambda ARN. The route key `ANY /{proxy+}` is a catch-all that passes every method and path to Lambda, which then runs the full Express.js router internally. This architecture means the Express application requires *zero code changes* to run inside Lambda — it was designed to run as a plain Node.js process and uses the same routing logic in both local Docker and production Lambda environments.

API Gateway auto-deploys every time the Lambda function is updated, and the stage name `$default` makes the base URL clean (no stage prefix in the path).

5.4 Lambda + DynamoDB — Data Persistence

The backend uses the **AWS SDK v3** DynamoDB Document Client (`@aws-sdk/lib-dynamodb`) with `marshallOptions: { removeUndefinedValues: true }` to automatically strip undefined fields before writing to DynamoDB. The client reads the region from `AWS_REGION` and table names from `TABLE_USERS`, `TABLE_PROJECTS`, and `TABLE_TASKS` environment variables injected by CloudFormation at deployment time.

The tasks table has a **Global Secondary Index** named `projectId-index` with `projectId` as the hash key and `createdAt` as the range key. This enables `QueryCommand` to efficiently fetch all tasks belonging to a specific project without performing a full table scan, which is critical for performance and cost at scale:

```

1 const { Items = [] } = await docClient.send(new QueryCommand({
2   TableName: TABLES.TASKS,
3   IndexName: "projectId-index",
4   KeyConditionExpression: "projectId = :pid",
5   ExpressionAttributeValues: { ":pid": req.params.projectId },
6 }));

```

Listing 5.1: GSI Query for project-scoped task retrieval (tasks.js)

5.5 Lambda + IAM — Least-Privilege Access

The Lambda execution role is defined entirely within CloudFormation. It attaches the AWS-managed `AWSLambdaBasicExecutionRole` for CloudWatch Logs access and an inline policy granting only the six DynamoDB actions the application needs (`GetItem`, `PutItem`, `UpdateItem`, `DeleteItem`, `Query`, `Scan`) scoped to the three table ARNs and the tasks GSI ARN. No wildcard resource permissions are used.

5.6 Lambda + CloudWatch — Observability

All `console.log` and `console.error` calls in the Express error handler are automatically forwarded to CloudWatch Logs via the Lambda execution environment. This provides request-level debugging without any additional instrumentation — cold-start durations, DynamoDB throttling errors, and upstream Ollama timeouts all surface as log entries in the Lambda log group.

5.7 AI Chat — Ollama Integration (Local Development)

The AI route (`/api/ai`) integrates with a locally running Ollama server. Before answering the user's question, the route handler performs three parallel DynamoDB `ScanCommand` calls to build a live **workspace snapshot**: a structured plain-text summary of every project, every task (with status, priority, and assignee name), and per-user task counts. This snapshot is injected as the system prompt, so the LLM can answer workspace-specific questions factually without hallucinating. The model is configured with `temperature: 0.3` for more factual, less creative responses.

6. Application Screenshots

This chapter presents screenshots of the application user interface demonstrating its key features and views.

6.1 Dashboard — Board View (Kanban)

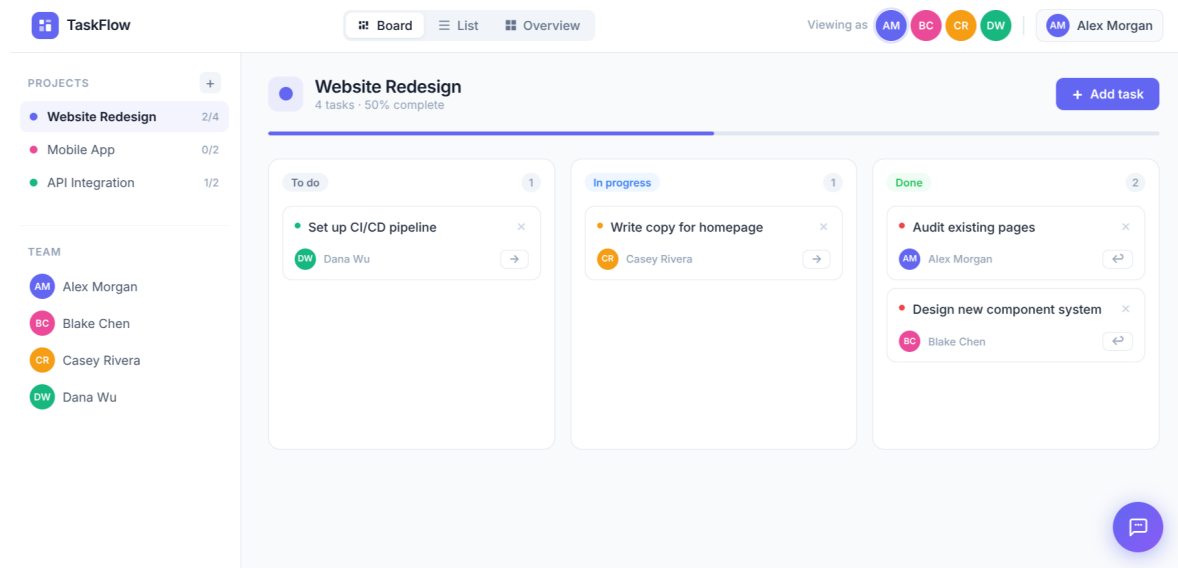


Figure 6.1: Board view: Kanban columns with task cards, priority badges, and team avatars

The Board view is the default landing screen. Each column corresponds to a task status. Task cards display the title, assigned team member avatar, and a colour-coded priority badge. Cards can be dragged between columns to update their status instantly via an optimistic UI update backed by a `PUT /api/tasks/:id` call.

6.2 Dashboard — List View

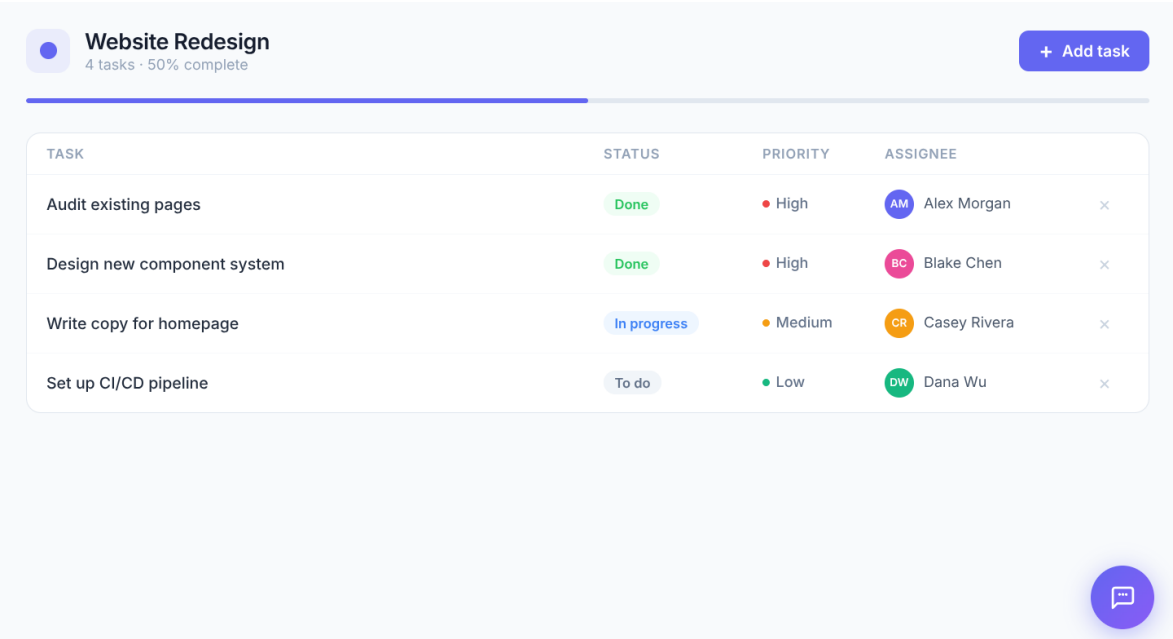


Figure 6.2: List view: tabular task display with status, priority, and assignee columns

The List view renders tasks in a structured table layout, making it easier to scan many tasks at once, sort by attribute, or perform bulk reviews in sprint planning sessions.

6.3 Dashboard — Overview

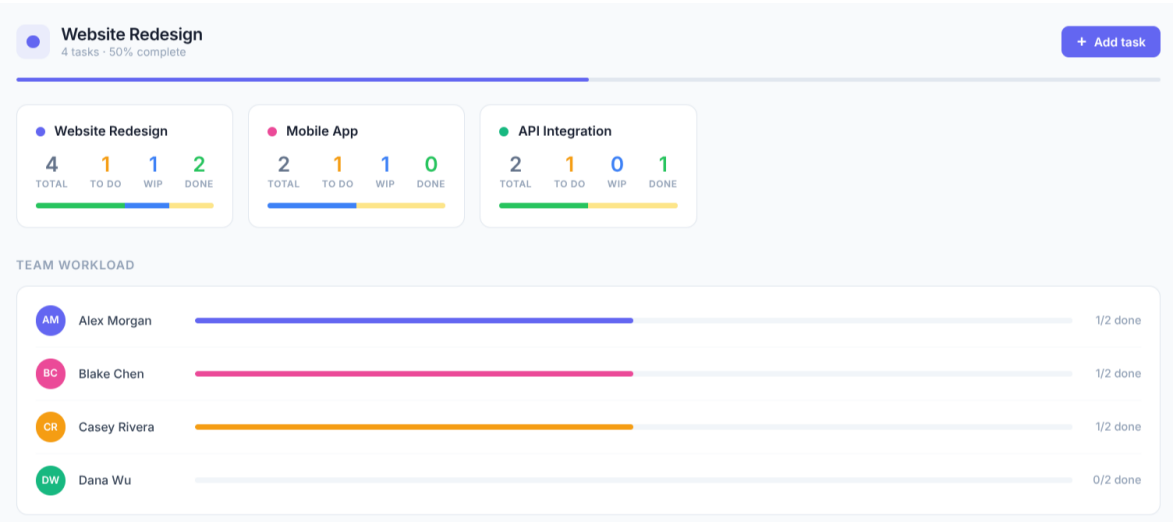


Figure 6.3: Overview: project-level completion statistics and team workload summary

The Overview presents aggregate metrics per project: total tasks, tasks done, tasks in progress, and completion percentage. It gives project leads a quick health check across all active projects without drilling into individual task boards.

6.4 Create Project Modal

PROJECTS

Project name...

● ● ● ● ● ● ●

●

Create Cancel

● Website Redesign 2/4

● Mobile App 0/2

● API Integration 1/2

Figure 6.4: Create project modal: form for adding a new project to the workspace

Projects are created through a modal form that POSTs to `/api/projects`. The new project is immediately reflected in the sidebar project list and the Board view switches to it automatically.

6.5 Task Detail Modal

New task ×

TASK NAME

What needs to be done?

PRIORITY

☒ Low ☐ Medium ☐ High

ASSIGN TO

☒ **AM** Alex Morgan ✓

☐ **BC** Blake Chen

☐ **CR** Casey Rivera

☐ **DW** Dana Wu

Create task **Cancel**

Figure 6.5: Task detail modal: editing title, status, priority, and assignee

Clicking any task card opens a detail modal that allows editing the title, status, priority, and assignee. Changes are persisted via `PUT /api/tasks/:id` using a `DynamoDB UpdateCommand` that sets only the modified attributes.

6.6 AI Chat Widget

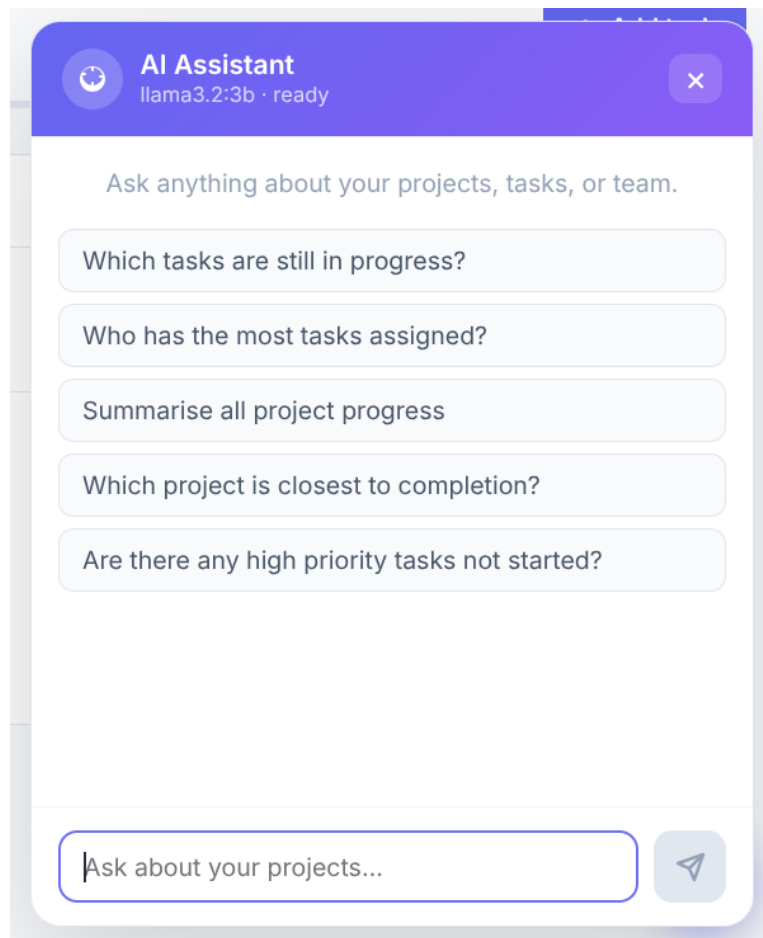


Figure 6.6: AI chat widget: Ollama-powered assistant with live workspace context injection

The floating chat button in the bottom-right corner opens the AI assistant. The backend builds a live workspace snapshot from DynamoDB and injects it into the system prompt before forwarding the conversation to the `llama3.2:3b` model running on the local Ollama server.

7. Code Overview

This chapter highlights the key source files that implement the application's core behaviour, complemented by code screenshots.

7.1 Express Server Entry Point (backend/src/app.js)

```
1  import express from "express";
2  import cors from "cors";
3  import usersRouter from "../routes/users.js";
4  import projectsRouter from "../routes/projects.js";
5  import tasksRouter from "../routes/tasks.js";
6  import aiRouter from "../routes/ai.js";
7
8  const app = express();
9  const PORT = process.env.PORT || 3001;
10
11 app.use(cors({ origin: process.env.CORS_ORIGIN || "*" }));
12 app.use(express.json());
13
14 // Health check
15 app.get("/health", (_, res) => res.json({ status: "ok", ts: new Date().toISOString() }));
16
17 // Routes
18 app.use("/api/users", usersRouter);
19 app.use("/api/projects", projectsRouter);
20 app.use("/api/tasks", tasksRouter);
21 app.use("/api/ai", aiRouter);
22
23 // Tasks nested under projects (shares same router, routes declared with prefix)
24 app.use("/api", tasksRouter);
25
26 // Error handler
27 app.use((err, req, res, _next) => {
28   console.error(err);
29   res.status(500).json({ error: err.message || "Internal server error" });
30 });
31
32 app.listen(PORT, () => console.log(`PixelPM API running on :${PORT}`));
```

Figure 7.1: backend/src/app.js: Express server, route mounting, and global error handler

The entry point registers CORS (origin from `CORS_ORIGIN` environment variable), JSON body parsing, four route modules, a health check endpoint at `/health`, and a global error handler that normalises all thrown errors to JSON 500 responses.

7.2 DynamoDB Client Configuration (backend/src/db/dynamodb.js)

```
1  import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2  import { DynamoDBDocumentClient } from "@aws-sdk/lib-dynamodb";
3
4  const clientConfig = {
5    region: process.env.AWS_REGION || "us-east-1",
6  };
7
8  // When running locally (Docker Compose), point at DynamoDB Local
9  if (process.env.DYNAMODB_ENDPOINT) {
10   clientConfig.endpoint = process.env.DYNAMODB_ENDPOINT;
11   clientConfig.credentials = {
12     accessKeyId: process.env.AWS_ACCESS_KEY_ID || "local",
13     secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY || "local",
14   };
15 }
16
17 const client = new DynamoDBClient(clientConfig);
18
19 export const docClient = DynamoDBDocumentClient.from(client, {
20   marshalOptions: { removeUndefinedValues: true },
21 });
22
23 export const TABLES = {
24   USERS: process.env.TABLE_USERS || "pm_users",
25   PROJECTS: process.env.TABLE_PROJECTS || "pm_projects",
26   TASKS: process.env.TABLE_TASKS || "pm_tasks",
27 };
```

Figure 7.2: backend/src/db/dynamodb.js: dual-mode DynamoDB client (AWS or local emulator)

The client module reads `DYNAMODB_ENDPOINT` to decide whether to point at AWS or DynamoDB Local, making the same codebase run unchanged in both environments. Table names are read from environment variables injected by Docker Compose or CloudFormation.

7.3 Tasks Route with GSI Query (backend/src/routes/tasks.js)

```

19 // GET /api/projects/:projectId/tasks
20 router.get("/projects/:projectId/tasks", async (req, res, next) => {
21   try {
22     const { Items = [] } = await docClient.send(new QueryCommand({
23       TableName: TABLES.TASKS,
24       IndexName: "projectId-index",
25       KeyConditionExpression: "projectId = :pid",
26       ExpressionAttributeValues: { ":pid": req.params.projectId },
27     }));
28     res.json(Items.map(norm).sort((a, b) => (a.createdAt || "").localeCompare(b.createdAt || "")));
29   } catch (e) { next(e); }
30 });
31
32 // POST /api/projects/:projectId/tasks
33 router.post("/projects/:projectId/tasks", async (req, res, next) => {
34   try {
35     const { title, priority = "medium", assignee, status = "todo" } = req.body;
36     if (!title?.trim()) return res.status(400).json({ error: "title is required" });
37
38     const task = {
39       taskId: uuidv4(),
40       projectId: req.params.projectId,
41       title: title.trim(),
42       priority,
43       assignee: assignee || null,
44       status,
45       createdAt: new Date().toISOString(),
46       updatedAt: new Date().toISOString(),
47     };
48
49     await docClient.send(new PutCommand({ TableName: TABLES.TASKS, Item: task }));
50     res.status(201).json(norm(task));
51   } catch (e) { next(e); }
52 });

```

Figure 7.3: backend/src/routes/tasks.js: RESTful task endpoints including GSI-backed project query

The tasks router demonstrates key DynamoDB patterns: `ScanCommand` for full table reads, `QueryCommand` against the GSI for project-scoped fetches, and a `UpdateCommand` that dynamically builds the update expression from only the fields present in the request body — avoiding unintentional attribute overwrites.

7.4 AI Route with Workspace Context Builder (backend/src/

```

64 // Body: { messages: [{ role: "user"|"assistant", content: string }] }
65 router.post("/chat", async (req, res, next) => {
66   try {
67     const { messages = [] } = req.body;
68     if (!messages.length) return res.status(400).json({ error: "messages array required" });
69
70     const context = await buildContext();
71
72     const systemPrompt = `You are a helpful project management assistant for PixelPM.
73     Use ONLY the workspace snapshot below to answer questions. Be concise and direct.
74     If asked something outside the workspace data, say you don't have that information.
75
76     === WORKSPACE SNAPSHOT ===
77     ${context}
78     =====`;
79
80     const body = {
81       model: MODEL,
82       messages: [{ role: "system", content: systemPrompt }, ...messages],
83       stream: false,
84       options: { temperature: 0.3 }, // lower temp = more factual
85     };
86
87     const ollamaRes = await fetch(`${OLLAMA}/api/chat`, {
88       method: "POST",
89       headers: { "Content-Type": "application/json" },
90       body: JSON.stringify(body),
91       signal: AbortSignal.timeout(120_000), // 2 min max
92     });
93
94     if (!ollamaRes.ok) {
95       const txt = await ollamaRes.text();
96       return res.status(502).json({ error: `Ollama: ${txt}` });
97     }
98
99     const data = await ollamaRes.json();
100     res.json({ message: data.message, model: data.model });
101   } catch (e) {
102     next(e);
103   }
104 });
105
106 export default router;

```

Figure 7.4: backend/src/routes/ai.js: live workspace snapshot builder and Ollama chat proxy

The AI route uses `Promise.all` to concurrently scan all three DynamoDB tables, then assembles a structured workspace snapshot that is injected as the LLM system prompt. This grounding technique ensures the model answers with real project data rather than hallucinated content.

7.5 React Application Component (src/App.jsx)

```

1  import { useState, useEffect } from "react";
2  import * as api from "../services/api.js";
3  import AiChat from "../components/AiChat.jsx";
4
5  const STATUSES = ["todo", "in-progress", "done"];
6  const STATUS_LABELS = { "todo": "To do", "in-progress": "In progress", "done": "Done" };
7  const PRIORITIES = ["low", "medium", "high"];
8  const PCOLORS = { low: "#10b981", medium: "#f59e0b", high: "#ef4444" };
9  const SBG_COLORS = { todo: "#f1f5f9", "in-progress": "#fff6ff", done: "#f0fdf4" };
10 const ST_COLORS = { todo: "#64748b", "in-progress": "#3b82f6", done: "#22c55e" };
11 const PALETTE = ["#6366f1", "#ec4899", "#f59e0b", "#10b981", "#3b82f6", "#8b5cf6", "#ef4444", "#14b8a6"];
12 const VIEW_ICONS = {
13   board: "M3 5h4v5H3V5Zm0 8h4v6H3v-6Zm6-8h4v3H9V5Zm0 6h4v8H9v-8Zm6-6h4v8h-4V5Zm0 11h4v3h-4v-3Z",
14   list: "M3 4h18v2H3V4Zm0 7h18v2H3v-2Zm0 7h18v2H3v-2Z",
15   overview: "M3 3h8v8H3V3Zm0 10h8v8H3v-8Zm10-10h8v8h-8V3Zm0 10h8v8h-8v-8Z",
16 };
17
18 export default function App() {
19   const [projects, setProjects] = useState([]);
20   const [users, setUsers] = useState([]);
21   const [loading, setLoading] = useState(true);
22   const [error, setError] = useState(null);
23   const [activeId, setActiveId] = useState(null);
24   const [currentUser, setCurrentUser] = useState(null);
25   const [view, setView] = useState("board");
26   const [showNewProj, setShowNewProj] = useState(false);
27   const [showNewTask, setShowNewTask] = useState(false);
28   const [newProjName, setNewProjName] = useState("");
29   const [newProjColor, setNewProjColor] = useState("#6366f1");
30   const [newTask, setNewTask] = useState({ title: "", priority: "medium", assignee: "" });
31   const [dragTask, setDragTask] = useState(null);
32   const [dragOver, setDragOver] = useState(null);

```

Figure 7.5: src/App.jsx: centralised React state, view modes, and drag-and-drop handler

The main React component is the central hub of frontend state. It maintains projects, tasks, users, and the active project selection using `useState` and `useEffect` hooks. The drag-and-drop handler performs an optimistic UI update (immediately moving the card in local state) before calling the API, so the interface feels instant even over variable network conditions.

7.6 CloudFormation Infrastructure Template (infrastructure/

```

133 # — API Gateway (HTTP API – cheaper than REST API, free tier: 1M req/month) –
134
135 ApiGateway:
136   Type: AWS::ApiGatewayV2::Api
137   Properties:
138     Name: !Sub "${AppName}-api"
139     ProtocolType: HTTP
140     CorsConfiguration:
141       AllowOrigins: ["*"]
142       AllowMethods: [GET, POST, PUT, DELETE, OPTIONS]
143       AllowHeaders: [Content-Type, Authorization]
144
145 ApiIntegration:
146   Type: AWS::ApiGatewayV2::Integration
147   Properties:
148     ApiId: !Ref ApiGateway
149     IntegrationType: AWS_PROXY
150     IntegrationUri: !Sub "arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${BackendLambda.Arn}/invocations"
151     PayloadFormatVersion: "2.0"
152
153 ApiRoute:
154   Type: AWS::ApiGatewayV2::Route
155   Properties:
156     ApiId: !Ref ApiGateway
157     RouteKey: "ANY /{proxy+}"
158     Target: !Sub "integrations/${ApiIntegration}"
159
160 ApiStage:
161   Type: AWS::ApiGatewayV2::Stage
162   Properties:
163     ApiId: !Ref ApiGateway
164     StageName: $default
165     AutoDeploy: true

```

Figure 7.6: infrastructure/cloudformation.yaml: complete IaC definition for all AWS resources

The CloudFormation template is the single source of truth for all AWS infrastructure. It provisions every resource — DynamoDB tables, IAM role, Lambda, API Gateway, S3, CloudFront OAC and distribution — and outputs the CloudFront URL, API Gateway URL, and table names for use in deployment scripts.

7.7 Docker Compose Local Stack (docker-compose.yml)

```
1  services:
2
3      # — DynamoDB Local —————
4      dynamodb:
5          image: amazon/dynamodb-local:latest
6          container_name: pixelpm-dynamodb
7          ports:
8              - "8000:8000"
9          command: ["-jar", "DynamoDBLocal.jar", "-sharedDb", "-inMemory"]
10
11     # — One-time setup: create tables + seed demo data —————
12     db-init:
13         image: node:20-alpine
14         container_name: pixelpm-db-init
15         working_dir: /app
16         volumes:
17             - ./infrastructure:/app
18         environment:
19             DYNAMODB_ENDPOINT: http://dynamodb:8000
20             AWS_REGION: us-east-1
21             AWS_ACCESS_KEY_ID: local
22             AWS_SECRET_ACCESS_KEY: local
23         command: >
24             sh -c "
25                 echo 'Waiting for DynamoDB...' &&
26                 until nc -z dynamodb 8000 2>/dev/null; do sleep 1; done &&
27                 echo 'DynamoDB ready.' &&
28                 npm install --silent &&
29                 npm run init
30             "
31         depends_on:
32             - dynamodb
33         restart: "no"
```

Figure 7.7: docker-compose.yml: six-container local development stack with health checks

The Docker Compose file orchestrates the full local stack with proper startup ordering: DynamoDB Local must be healthy before the backend starts, the backend must pass its health check before the frontend starts, and the model-init job ensures the llama3.2:3b model is available before AI chat requests are served.

8. Conclusion

Pixelpm demonstrates that a production-grade project management tool can be built and operated entirely within the AWS free tier. By combining serverless compute (Lambda), managed NoSQL storage (DynamoDB), a CDN-backed static frontend (S3 + CloudFront), and an HTTP API (API Gateway), the application achieves high availability and low latency with zero operational overhead and effectively zero cost at small team scales.

The architecture has several noteworthy properties:

- **Infrastructure as Code** — every AWS resource is defined in a single CloudFormation template, making the environment fully reproducible and diff-able in version control.
- **Dual-mode data access** — the DynamoDB client transparently switches between AWS and the local emulator based on a single environment variable, enabling a realistic offline development experience with no code changes.
- **Free-tier by design** — PAY_PER_REQUEST DynamoDB, Lambda’s generous free invocation quota, and CloudFront’s always-free CDN tier mean a team of 10–15 people will never exceed the free tier under normal usage.
- **AI without external APIs** — by routing AI chat through Ollama, the application preserves data privacy (workspace data never leaves the local machine) and incurs no per-token API costs.
- **Least-privilege security** — the IAM role grants exactly the six DynamoDB actions required, scoped to exactly the three table ARNs, with no wildcard permissions.

The project serves as both a practical productivity tool and a reference implementation of modern serverless architecture patterns on AWS.